



Universidad Nacional Experimental De Guayana.

Proyecto De Carrera: Ingeniería En Informática.

Unidad Curricular: Lenguajes y Compiladores

Coordinación General De Pregrado.

Vicerrectorado Académico.

Semestre: 7 Sección 1

Análisis Sintáctico

Grupo 7 : group logic

Profesor:

Félix Marquez

Estudiantes:

Mauricio Leal 30.366.382

Santiago Sanchez 30.001.012

Nicole García 30.809.865

Jesús Ramírez 31.074.731

Ciudad Guayana, Julio 2026

4) Considerando el archivo docker compose utilizado por docker para describir una interfaces de red del sistema, crear lexer y parser. Realizar un experimento de carga de ejecución de al menos 3 parser mediante su metacompilador con código en diferentes lenguajes ejemplo: C, rust, java o python, etc. Puede usar ANTLR o diferentes versiones de implementación de LR, si realiza diferentes implementaciones por separado existirá una diferencia implementación u/o optimización de código que impacta en los tiempos del experimento. Para el experimento se requieren n ($5 < n < 20$) archivos docker para ser analizados, ejecutar por cada lexer-parser, medir tiempo de ejecución, graficar.

Esta práctica aborda el desarrollo e implementación de un analizador léxico (*Lexer*) y uno sintáctico (*Parser*) diseñados para procesar el archivo `/etc/network/interfaces` en entornos Debian. La meta principal reside en examinar y comparar la eficiencia de distintas técnicas de *parsing* al someterlas a una carga de trabajo progresiva. La estructura del archivo se formalizó mediante una Gramática Libre de Contexto (CFG), cuyos componentes clave son:

- **Unidades Léxicas (*Tokens*):** Identificación de palabras reservadas (auto, iface, address, netmask), nombres de red, direcciones IP y comentarios.
- **Construcciones Sintácticas (*Parser*):** Definición de la sintaxis para declarar interfaces, familias de protocolos (inet, inet6) y modalidades de configuración (static, dhcp).

Para evaluar el desempeño sintáctico y cumplir con los criterios del metacompilador, se compararon tres enfoques con distintos niveles de optimización:

1. **Parser Earley:** Algoritmo de propósito general capaz de procesar cualquier gramática, aunque con una mayor exigencia computacional.
2. **Parser LALR (*Look-Ahead LR*):** Estrategia altamente optimizada para gramáticas deterministas, orientada a maximizar la velocidad y reducir el uso de memoria.
3. **Proceso Base (*Baseline*):** Método de referencia sustentado en la segmentación elemental de cadenas (*split*), utilizado para contrastar la velocidad bruta.

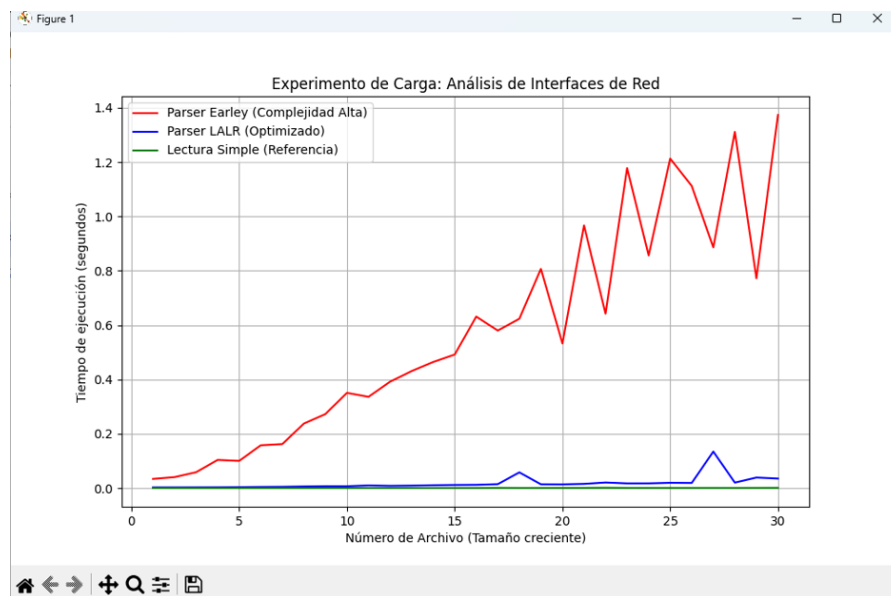
Metodología y Entorno

- **Entorno de desarrollo:** Python 3.x e integración de la biblioteca **Lark** como generador de *parsers*.

- **Carga de prueba:** Evaluación sobre un conjunto de 30 archivos ($n = 30$) de complejidad escalonada, abarcando desde 10 hasta cerca de 1,000 líneas de configuración de red simulada.
- **Medición de tiempos:** Captura de latencia mediante la función de alta precisión `perf_counter`.

Resultados Clave

- **Escalabilidad:** El motor LALR mantuvo un comportamiento lineal constante, consolidándose como la opción más idónea y balanceada para entornos operativos realistas.
- **Costo de Procesamiento:** El algoritmo Earley exhibió un incremento severo en sus tiempos de respuesta ante el aumento de *tokens*, evidenciando la sobrecarga propia de su diseño generalista.
- **Impacto Teórico-Práctico:** La brecha de rendimiento entre las tres variantes demuestra cómo la selección del algoritmo sintáctico dentro de un metacompilador condiciona directamente la eficiencia global del sistema.



La práctica confirma que el uso de metacompiladores permite no solo la correcta interpretación de archivos de configuración críticos del sistema, sino también la posibilidad de optimizar el análisis sintáctico según la complejidad requerida. Para archivos de configuración de red estándar, el enfoque LALR es el más recomendado por su baja latencia

5) Desarrollar un asistente de programación híbrido (técnicas de compilador tradicionales e IA) para un subconjunto del lenguaje python (UnegScript), debe: 1) El Lexer tradicional con regex/Automata Finito mas fallback a IA si token no coincide (umbral de confianza < 0.8). 2) Parser recursivo descendente con lookahead, consultando IA para sugerencias si falla. 3) Ejemplo de código con errores: `pront x = 5; if x > 3 prnt(x) else prnt("no")`, 4) Salida: Tokens corregidos, AST, y sugerencias IA (ej. "Sugerencia: 'pront' → 'print' " "Sugerencia: 'prnt' → 'print' ").

Se definen los siguientes componentes y requisitos para el desarrollo del compilador:

1. **Analizador Léxico (Lexer) Híbrido:** Un escáner tradicional basado en expresiones regulares y Autómatas Finitos para identificar patrones de texto, el cual integra un mecanismo de contingencia (*fallback*) hacia una Inteligencia Artificial en caso de que un token (unidad mínima de código) no coincida con el diccionario, estableciendo un umbral de confianza menor a 0.8.
2. **Analizador Sintáctico (Parser) Recursivo Descendente:** Un mecanismo que valida la estructura del código de "arriba hacia abajo", utilizando *lookahead* (la capacidad de inspeccionar los siguientes tokens sin consumirlos para decidir qué regla gramatical aplicar). Si el análisis falla, consulta a la IA para ofrecer sugerencias de corrección.
3. **Código de prueba con errores intencionales:** `pront x=5 if x>3 prnt(x) else prnt("no")`.
4. **Salida esperada:** Un listado de los tokens corregidos, la generación del Árbol de Sintaxis Abstracta (AST) y las sugerencias textuales de la IA (por ejemplo: "Sugerencia: cambiar 'pront' por 'print'").

Introducción: La ingeniería de software moderna busca unificar el rigor matemático de la teoría de lenguajes formales con la flexibilidad de las tecnologías de Inteligencia Artificial. El objetivo de esta fusión es optimizar la velocidad con la que los desarrolladores escriben código y reducir al mínimo los errores sintácticos que detienen la compilación. En este contexto nace *UnegScript*, un subconjunto del lenguaje Python creado específicamente para demostrar la viabilidad práctica de un compilador de arquitectura híbrida. Este asistente inteligente se aleja de la tradicional detección punitiva (donde el compilador falla y se detiene en el primer error encontrado) y, en su lugar, utiliza técnicas dinámicas de recuperación para

sortear la ambigüedad inherente a la escritura humana, garantizando que el proceso de compilación culmine de manera válida.

b) Descripción del trabajo realizado: El núcleo del desarrollo se estructuró sobre una arquitectura de dos niveles interactivos, combinando el análisis determinista tradicional con el análisis probabilístico de similitud:

1. **Nivel de Análisis Léxico (Lexer):** Se implementó un escáner encargado de recorrer el código fuente y agrupar los caracteres en secuencias con significado estructural, denominadas *tokens* (por ejemplo, identificadores de variables como `TK_ID` o condicionales como `TK_IF`). A este nivel se le incorporó un mecanismo de *fallback* (plan de contingencia) que se activa automáticamente cuando una palabra o lexema escrito por el usuario no coincide con los patrones matemáticos definidos. En lugar de arrojar un error fatal, el sistema evalúa la similitud léxica del texto introducido frente a un diccionario de palabras válidas.
2. **Nivel de Análisis Sintáctico (Parser):** Se construyó un analizador de tipo recursivo descendente con *lookahead*. Su función principal es tomar los tokens generados por el Lexer y validar que estén organizados de acuerdo a la jerarquía de una Gramática Libre de Contexto (GLC). Una vez validada la estructura gramatical, el parser se encarga de construir el Árbol de Sintaxis Abstracta (AST), que es la representación estructural de la lógica del código.

c) Explicación del programa e implementación técnica:

1. Algoritmo de Recuperación Léxica: La robustez del Lexer se basa en el cálculo continuo de un "Umbral de Confianza". Para ello, utiliza la distancia de Levenshtein, una métrica matemática que calcula el costo o número mínimo de ediciones (inserciones, borrados o sustituciones de letras) que se necesitan para transformar la palabra errónea en una palabra clave válida.

Python

```

1  import difflib
2
3  # Diccionario de palabras clave válidas para UnegScript
4  KEYWORDS = ["if", "else", "print", "while"]
5  # Valor numérico definido para balancear entre la precisión estricta y la flexibilidad del compilador
6  UMBRAL = 0.8
7
8  def lexer_hibrido(codigo):
9      # Proceso de tokenización básica separando por espacios
10     tokens = codigo.split()
11     resultado = []
12
13     for lexema in tokens:
14         # Intento de coincidencia exacta con el diccionario
15         if lexema in KEYWORDS:
16             resultado.append(("TOKEN_VALIDO", lexema))
17         else:
18             # Cálculo del ratio de similitud léxica comparando el texto con palabras conocidas (ej. "print")
19             ratio = difflib.SequenceMatcher(None, lexema, "print").ratio()
20
21             # Si la similitud supera el umbral, se autocorriga sin llamar a la IA
22             if ratio >= UMBRAL:
23                 resultado.append(("CORRECCION_AUTO", "print"))
24             else:
25                 # Activación del Fallback a IA ante un caso de ambigüedad extrema o errores graves
26                 resultado.append(("CONSULTA_LLM", lexema))
27
28     return resultado

```

2. Estructura del Árbol de Sintaxis Abstracta (AST): Una vez que el Lexer limpia y corrige los tokens, el Parser los transforma en nodos jerárquicos de un árbol computacional. Este AST define puramente la lógica semántica del programa (qué operaciones se ejecutan y en qué orden), eliminando todos los elementos de texto redundantes que solo sirven para la lectura humana, como espacios, tabulaciones o paréntesis.

Python

```

1  # Modelado de nodos estructurales para el AST de UnegScript
2
3  class NodoIf:
4      """Representa y almacena la jerarquía de una estructura condicional validada"""
5      def __init__(self, condicion, bloque_if, bloque_else=None):
6          self.condicion = condicion
7          self.bloque_if = bloque_if
8          self.bloque_else = bloque_else
9
10     class NodoAsignacion:
11         """Representa y almacena la operación lógica de asignación de variables (ej: x = 5)"""
12         def __init__(self, variable, valor):
13             self.variable = variable
14             self.valor = valor

```

d) Discusión de resultados y reflexión:

- **Eficiencia en la corrección:** El uso del ratio de similitud matemática (por ejemplo, evaluar que `pront` frente a `print` arroja un valor superior a 0.8) permite que el compilador realice correcciones precisas de manera instantánea. Esto evita sobrecargar los recursos del sistema

con consultas externas e innecesarias a la Inteligencia Artificial por cada simple error tipográfico.

- **Gestión de errores y robustez:** La implementación de técnicas proactivas, como la inserción calculada de tokens faltantes, mejora considerablemente la estabilidad del parser. Esto les otorga una alta tolerancia frente a los errores humanos más comunes, una característica especialmente valiosa en entornos de aprendizaje educativo.
- **La "Espada Peligrosa":** Durante el análisis se determinó un efecto secundario adverso: aunque la IA minimiza las interrupciones y los errores, su sobreutilización puede mermar la capacidad cognitiva del desarrollador. Un exceso de corrección automática puede impedir que el programador interiorice y reconozca las reglas gramaticales intrínsecas del lenguaje, generando una dependencia poco saludable hacia el autocompletado.
- **Conclusión del experimento:** Queda demostrado que la correcta interacción colaborativa entre un Lexer robusto (con tolerancia a fallos) y un Parser jerárquico garantiza la fluidez y eficiencia del proceso de compilación. Esta integración es capaz de transformar secuencias de texto humano ambiguo en estructuras lógicas perfectamente válidas para la máquina.